

SmartReco — Product Requirements Document

Behavioral AI Recommendation Agent

Submission for the **SmartReco Build Challenge 2026** · Krish Naik Academy × Mesh API

Stack: FastAPI · LangGraph · Next.js · PostgreSQL · Qdrant · Redis · Mesh API

Deadline: 11 August 2026

1. Thesis — why this submission wins

The brief will produce a hundred submissions that do the same thing: log a few clicks, stuff them into a prompt, ask an LLM for products, print the answer. Those submissions fail the same four ways — the vector DB drifts out of sync with SQL, the LLM invents courses that aren't in the catalog, an LLM call fires on every page view, and the "agent" is a single `chat.completions.create`. SmartReco is engineered specifically against those four failure modes.

Common failure mode	SmartReco's answer
Vector DB silently diverges from SQL	Transactional outbox — the dual-write is a committed DB row, replayed by a worker with retries. Sync is provable, and an admin endpoint reports drift.
LLM hallucinates products not in the catalog	Grounding validator node — generated <code>product_id</code> s must be a subset of retrieved candidates or the graph loops back. Enforced structurally, not by prompt-begging.
An LLM call per user action	Interest-drift trigger + three-layer cache — regeneration fires only when the user's interest profile actually moves, behind a cooldown. Target: under 1 LLM generation per 40 events .
"Agent" = one prompt	Seven-node LangGraph with conditional edges, retrieval self-grading, query refinement, and a bounded retry loop — traced end to end in LangSmith.

Three further things almost nobody ships in four days: an **offline evaluation harness** that scores our own recommendations, a **cost and latency dashboard** proving the efficiency claims, and **per-node model routing** through Mesh (cheap model for grading, strong model for persuasion).

README positioning line: *Most recommenders guess. SmartReco observes, grades its own retrieval, and refuses to recommend anything it can't point to in the catalog.*

2. Goals and non-goals

Goals

1. A working platform where a user browses a course catalog and receives recommendations that visibly change as their behavior changes.
2. Every recommendation is **grounded** — traceable to a specific catalog row and to the specific events that motivated it.
3. Every LLM/AI call routes through **Mesh API** (mandatory; a violation invalidates the submission).
4. Tracking that is provably non-blocking, batched, and lossless across tab close.
5. Demonstrable production thinking: caching, triggers, retries, observability, cost accounting.

Non-goals — state these explicitly in the README

- Payments, checkout, real course content or video delivery.
- OAuth or social login. The brief says *keep auth simple*, so: email + password with bcrypt and JWT.
- Multi-tenancy, horizontal scaling, Kubernetes.
- A/B testing infrastructure. We describe the hook; we don't build the harness.

Non-negotiable constraints

- Backend must be Python (FastAPI).
- All LLM calls must go through Mesh API at <https://api.meshapi.ai/v1>.
- Public GitHub repo, `.env` gitignored, no committed secrets.
- `requirements.txt` must list a web framework (`fastapi`) and an LLM client (`openai`) — this is a **critical CI check**.

Frontend note. The brief *suggests* Jinja2 but only *requires* a Python backend, so Next.js is compliant. Keep it in `/frontend` so the repo still reads as Python-first, and keep `requirements.txt` at the repo root where the checker looks. Add a README line explaining the swap so a human judge doesn't read it as ignoring the spec: *"Next.js chosen for the tracking SDK's reliability (Web Worker + sendBeacon); backend is FastAPI per the requirement."*

3. Personas and user stories

Priya — learner (primary). Browses courses, searches, lingers on advanced agentic-AI material. Wants to be told what to learn next in a way that feels written for her.

Arun — admin. Curates the catalog. Needs CRUD that never leaves the two stores inconsistent, and visibility into what the agent is doing.

#	Story	Acceptance criteria
U1	Sign up and log in	bcrypt-hashed password; JWT in httpOnly cookie; role claim in token
U2	Browse and search the catalog	Search is semantic with keyword fallback; every search is an event
U3	See a personalized recommendation block	Narrative plus 3-5 products, each with a "why this" line, on /dashboard
U4	Recommendations refresh as I explore	After a meaningful behavior shift the block updates within one trigger cycle; UI shows "Updated just now · based on your last 14 actions"
U5	Receive a daily digest email	Sent by APScheduler at a configured hour, only if the user had activity that day
A1	Create, edit, delete a product	Change lands in Postgres and Qdrant; outbox row marked done ; drift check returns zero
A2	Inspect agent runs	Table of runs: user, trigger reason, nodes visited, retrieval score, tokens, cost, latency, LangSmith link
A3	See system health	Event ingest rate, cache hit ratio, LLM calls per 100 events, outbox lag

4. System architecture

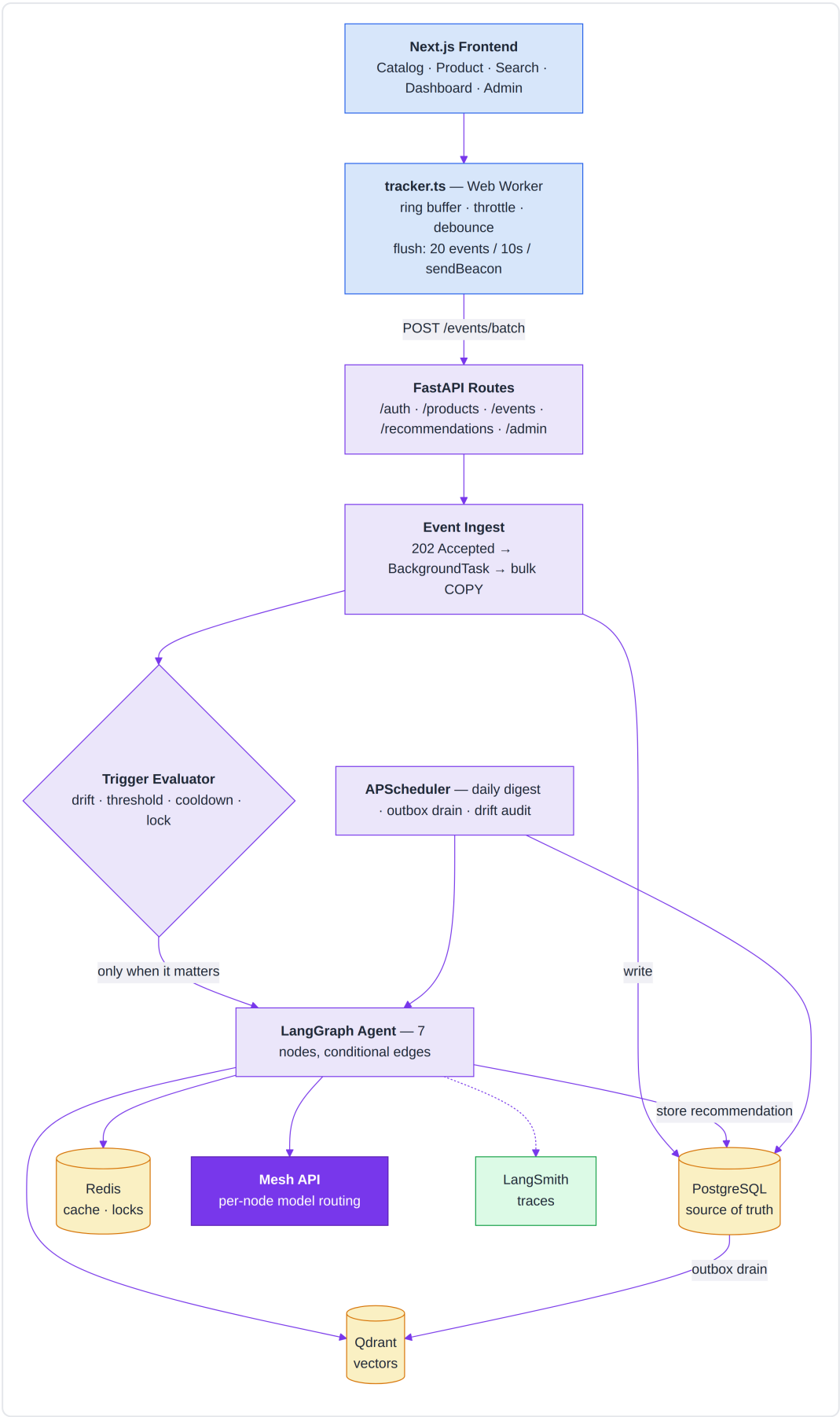


Figure 1 — System architecture

Why these choices — put this reasoning in the README, judges reward it.

- **Postgres over SQLite** — we need `JSONB` for event payloads, partial indexes, and concurrent writes from the scheduler and API. Ship a `docker-compose.yml` so setup stays one command.
 - **Qdrant over Chroma** — payload filtering and hybrid search are first-class and it persists cleanly in Docker. Chroma is an acceptable fallback if time runs short; keep the retriever behind an interface.
 - **Redis** — semantic cache, a distributed lock so two triggers can't run the agent for the same user concurrently, and rate limiting.
-

5. Data model

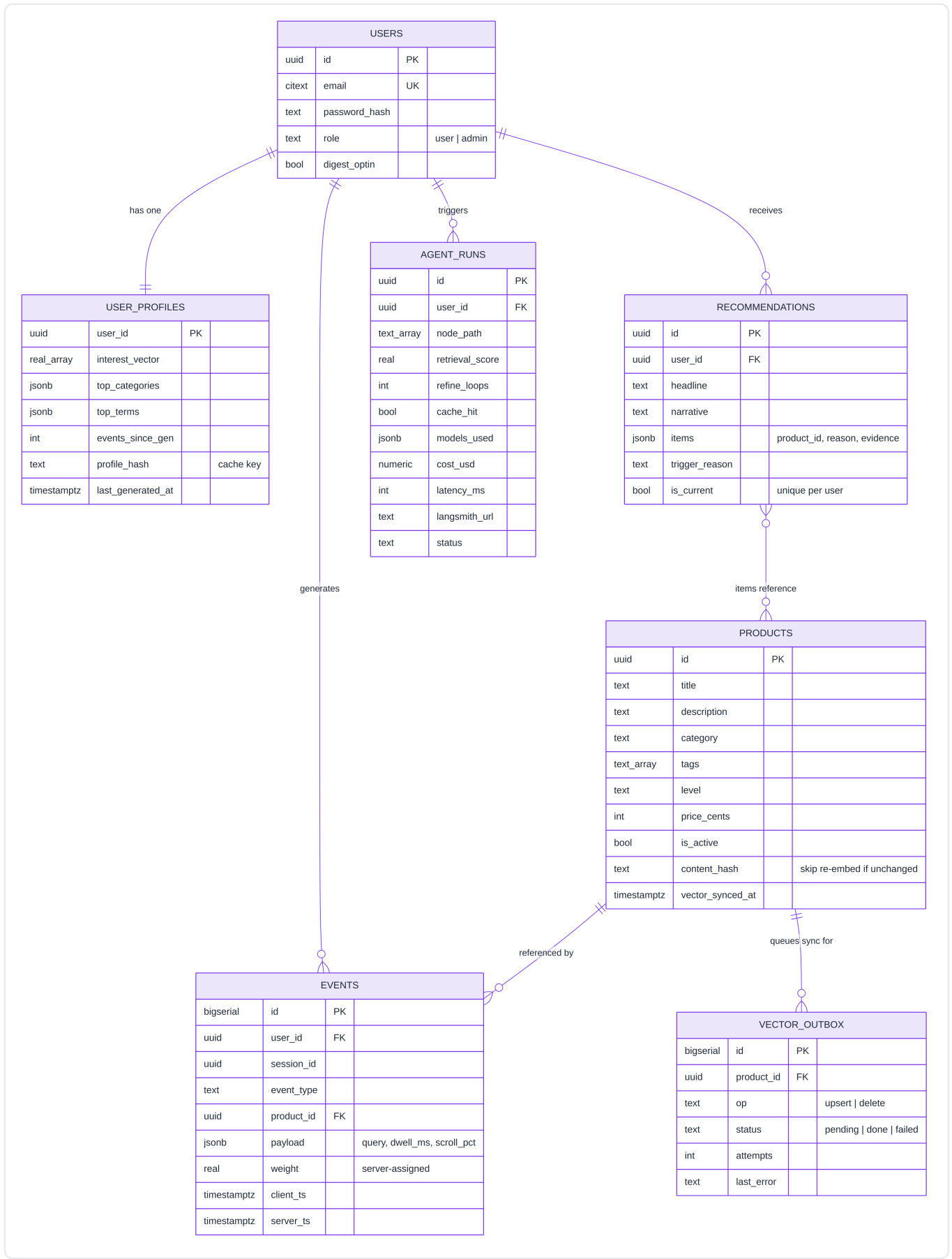


Figure 2 — Entity relationship model

Key DDL details

```
-- Idempotency: a retried batch cannot double-count
CREATE TABLE events (
  id          BIGSERIAL PRIMARY KEY,
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  session_id  UUID NOT NULL,
  event_type  TEXT NOT NULL,
  product_id  UUID REFERENCES products(id) ON DELETE SET NULL,
  payload     JSONB NOT NULL DEFAULT '{}',
  weight      REAL NOT NULL DEFAULT 1.0,
  client_ts   TIMESTAMPTZ NOT NULL,
  server_ts   TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (user_id, session_id, event_type, client_ts)
);
CREATE INDEX ON events (user_id, server_ts DESC);
CREATE INDEX ON events (user_id) INCLUDE (event_type, product_id)
  WHERE server_ts > now() - INTERVAL '7 days';

-- The outbox is what makes dual-write reliable rather than best-effort
CREATE TABLE vector_outbox (
  id          BIGSERIAL PRIMARY KEY,
  product_id  UUID NOT NULL,
  op          TEXT NOT NULL CHECK (op IN ('upsert','delete')),
  payload     JSONB NOT NULL,
  status      TEXT NOT NULL DEFAULT 'pending'
  CHECK (status IN ('pending','done','failed')),
  attempts    INTEGER NOT NULL DEFAULT 0,
  last_error  TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  processed_at TIMESTAMPTZ
);
CREATE INDEX ON vector_outbox (status, id) WHERE status = 'pending';

-- Exactly one current recommendation per user
CREATE UNIQUE INDEX ON recommendations (user_id) WHERE is_current;
```

Qdrant collection `products` : cosine distance, payload `{product_id, title, category, tags, level, price_cents, is_active}`. Point ID equals `product_id`, so upsert is naturally idempotent.

6. Feature specifications

6.1 Authentication (F1)

Email and password, bcrypt at cost 12, JWT (HS256, 24h) in an httpOnly SameSite=Lax cookie, `role` in claims. A FastAPI dependency `require_role("admin")` guards admin routes.

The seed script creates one admin and three demo users with **pre-baked divergent behavior histories**. This makes the demo instantly convincing: you can show three users receiving three genuinely different narratives without clicking for ten minutes.

6.2 Catalog and reliable dual-write (F2)

Never call Qdrant inside the request handler. Instead:

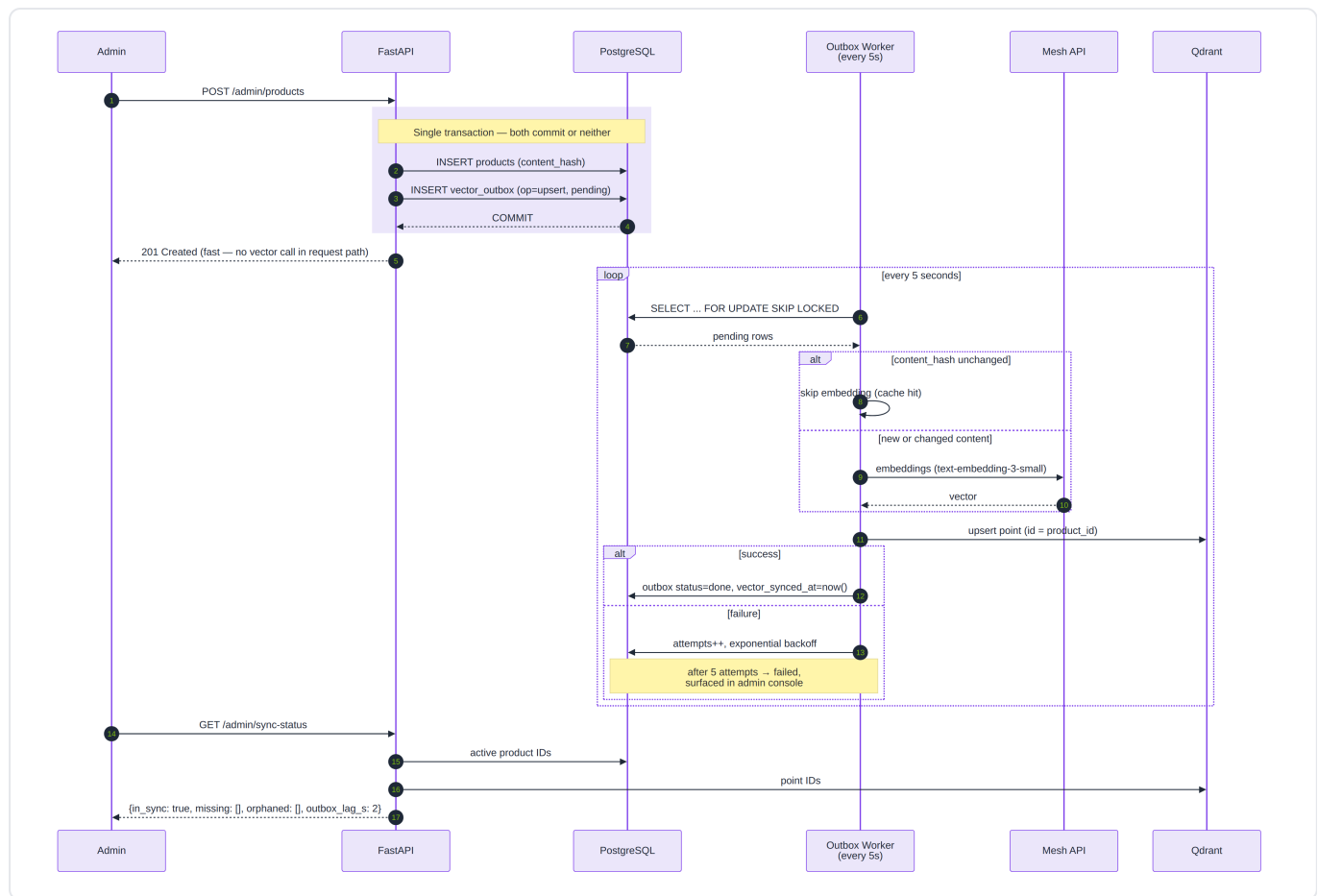


Figure 3 — Transactional outbox — reliable dual-write

Embedding skip. On update, recompute `content_hash`. If unchanged, skip re-embedding entirely — a small, visible cost optimization worth one README bullet.

Delete. Soft-delete in SQL (`is_active = false`) plus hard delete of the Qdrant point, both routed through the outbox.

Drift audit. `GET /api/admin/sync-status` compares Postgres active-product IDs against Qdrant point IDs and returns `{in_sync, missing_in_vector, orphaned_in_vector, outbox_lag_seconds}`. **Show this endpoint returning `in_sync: true` in the demo video.** It is the fastest way to prove to a judge that the dual-write is real rather than aspirational.

6.3 Behavioral tracking (F3)

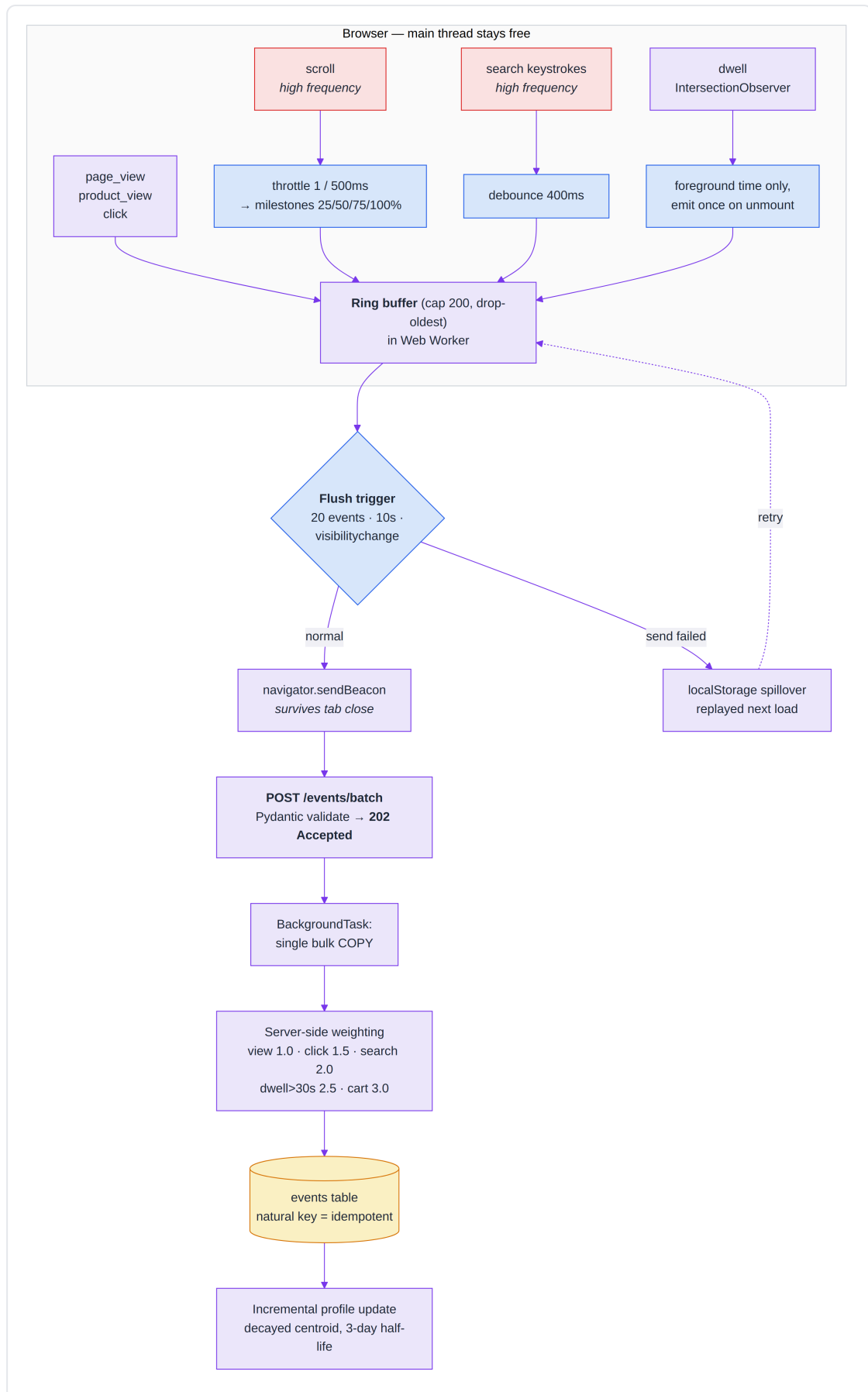


Figure 4 — Behavioral event tracking pipeline

Concern	Implementation
Never block the UI	All queue work in a Web Worker; zero synchronous XHR
Batching	Ring buffer capped at 200, drop-oldest; flush at 20 events or 10 seconds, whichever comes first
High-frequency events	Scroll throttled to one per 500 ms and reduced to milestone depths; search debounced 400 ms so keystrokes don't each become an event
Dwell time	<code>IntersectionObserver</code> plus <code>visibilitychange</code> ; accumulate foreground time only; emit one event on unmount
Tab close	<code>visibilitychange → hidden</code> fires <code>navigator.sendBeacon</code> , which survives navigation where <code>fetch</code> dies
Offline or failure	Failed batches return to the buffer; <code>localStorage</code> spillover replayed on next load
Idempotency	Client-generated event UUID plus the table's natural key prevents double-counting on retry
Trust	The server assigns <code>weight</code> by event type, so interest scoring never trusts the client

Prove it. Include a Playwright or k6 script that fires 1,000 events and asserts p95 client-side handler time under 2 ms with zero dropped events. One chart in the README. Nobody else will do this.

6.4 The LangGraph agent (F4) — the centerpiece

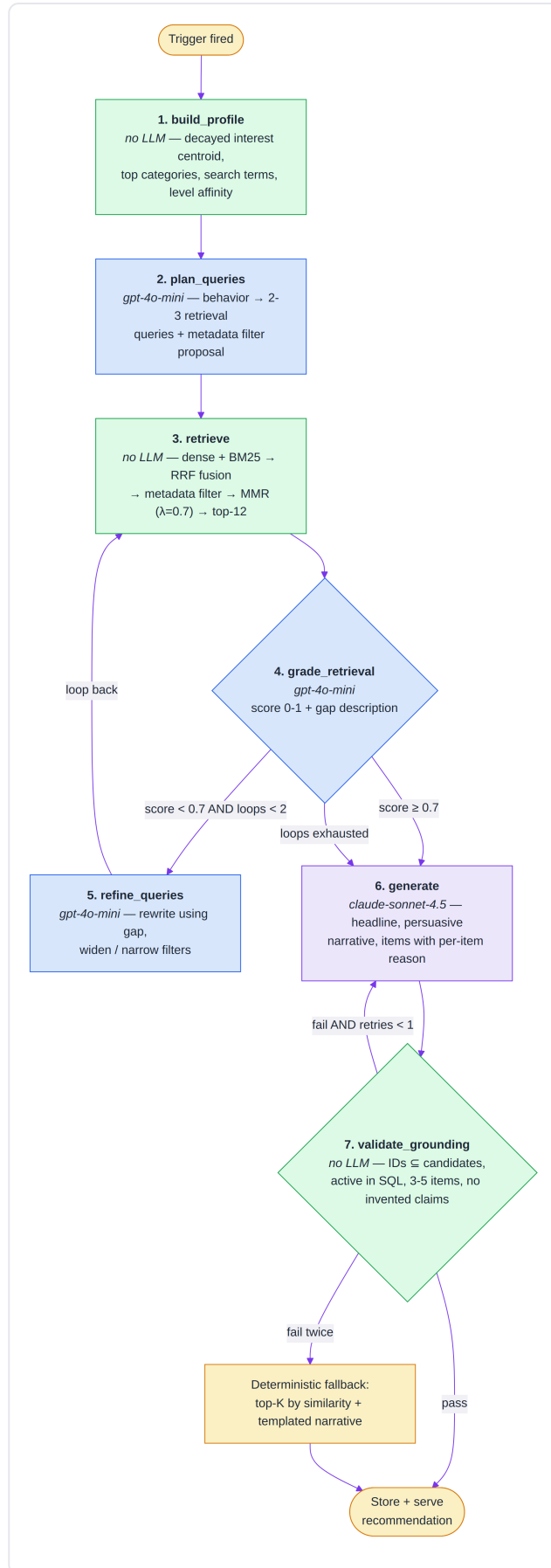


Figure 5 — LangGraph agent — nodes and conditional edges

```

class AgentState(TypedDict):
    user_id: str
    profile: UserProfile          # categories, terms, level, recent products
    evidence: list[Event]         # the specific events driving this run
    queries: list[str]           # planned retrieval queries
    candidates: list[Product]
    retrieval_score: float       # 0-1 self-grade
    refine_loops: int
    draft: RecommendationDraft | None
    validation_errors: list[str]
    node_path: list[str]

```

Node responsibilities

1. **build_profile** (no LLM) — reads recent events plus **user_profiles** and produces a compact behavioral summary: weighted top categories, search terms, level affinity, products already seen. Deterministic and free.
2. **plan_queries** (cheap model) — turns behavior into two or three *retrieval queries*, not a recommendation, plus a metadata filter proposal (**level=advanced** , category set, exclude already-purchased).
3. **retrieve** (no LLM) — per query: dense vector search plus BM25 keyword, fused with Reciprocal Rank Fusion, metadata-filtered, then MMR at $\lambda=0.7$ for diversity so you don't return five near-identical courses. Top 12 candidates.
4. **grade_retrieval** (cheap model, structured output) — scores 0–1: do these candidates actually serve this user's demonstrated interest? Returns per-candidate keep/drop plus a gap description. Conditional edge: score ≥ 0.7 goes to generate; below that with loops remaining goes to refine; otherwise generate with a hedged, exploratory framing.
5. **refine_queries** (cheap model) — rewrites queries using the grader's gap description, widens or narrows filters, loops back to retrieve. **This loop is exactly what the brief's bonus asks for — make it visible in the LangSmith trace and screenshot it in the README.**
6. **generate** (strong model, structured output) — produces **{headline, narrative, items: [{product_id, reason}]}**. The narrative must reference actual behavior ("you've come back to agentic-AI material three times this week and lingered longest on the advanced track"). Each item's reason ties one product to one observed signal.
7. **validate_grounding** (no LLM) — the hard gate. Every **product_id** must be in the retrieved candidate set, must exist and be active in Postgres; 3–5 items, no duplicates; no price or claim in the narrative absent from catalog metadata. On failure, one retry with errors injected; on a second failure, a deterministic top-K fallback. **The system never shows an ungrounded recommendation and never shows nothing.**

Runs use a LangGraph Postgres checkpointer so they are resumable and inspectable, and the full **node_path** is written to **agent_runs** .

6.5 Efficiency and trigger policy (F5) — explicitly judged

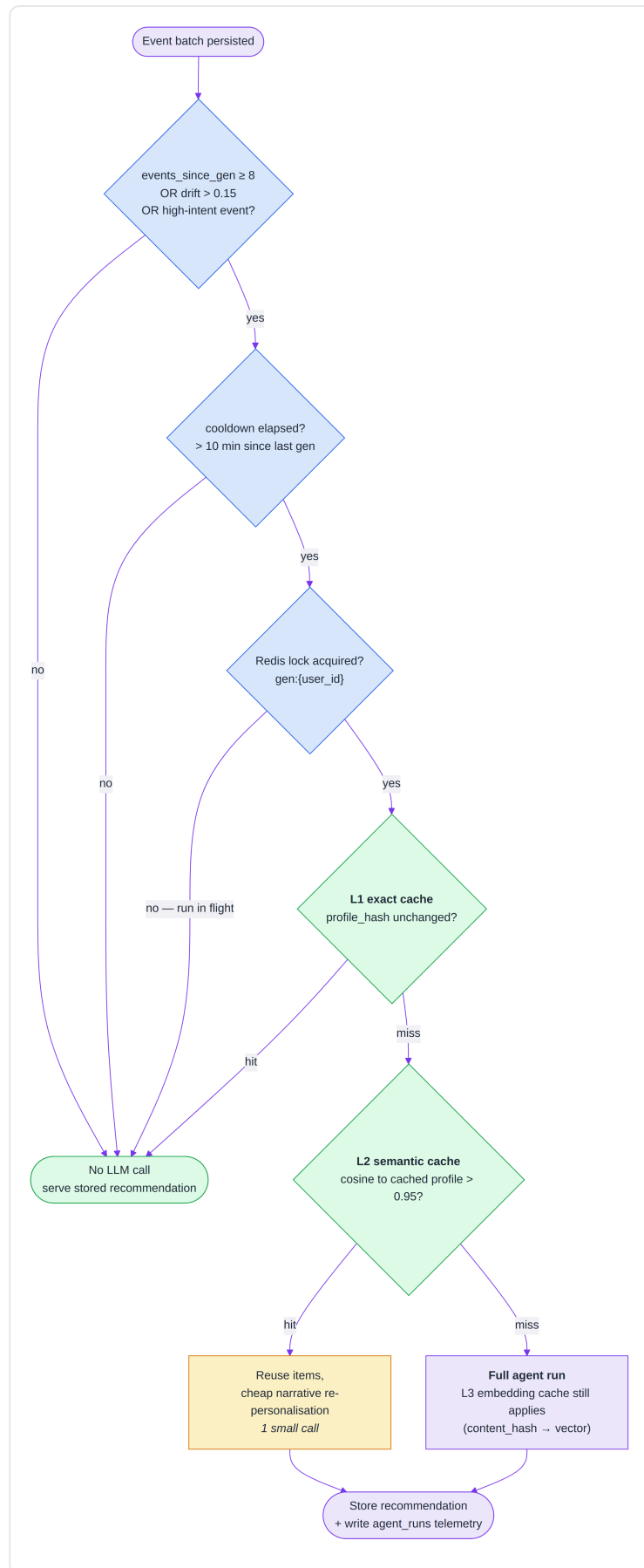


Figure 6 — Trigger policy and three-layer cache

Per-node model routing through Mesh. This is where Mesh's access to 1000+ models becomes a feature rather than a checkbox. Mesh exposes an `is_free` flag on every model and a dedicated free-tier filter, so the routing table is expressed as a *tier preference* rather than a fixed model ID.

Node	Tier	Resolution order	Rationale
<code>plan_queries</code>	free	free-tier instruct model → <code>gpt-4o-mini</code>	Short structured output; a small open-weight model is sufficient
<code>grade_retrieval</code>	free	free-tier instruct model → <code>gpt-4o-mini</code>	Binary/scalar classification; the cheapest capable model wins
<code>rerank</code>	free	free-tier instruct model → <code>gpt-4o-mini</code>	Pairwise scoring, no prose required
<code>generate</code>	quality	free-tier large model → <code>claude-sonnet-4.5</code>	The only node where persuasion quality justifies paid tokens
Embeddings	cheap	<code>openai/text-embedding-3-small</code>	1536-dim; embedding a 40-item catalog once costs cents

All of this lives in `settings.py`. **Say in the README that swapping models is a one-line change because Mesh is a single gateway** — it is both a real architectural point and a compliment to the sponsor.

Publish the numbers. A README table with LLM calls per 100 events, cache hit rate, median cost per recommendation, and p95 agent latency. Measured, not claimed.

6.5.1 Running at (near) zero cost — without breaking the rules

Constraint first. Rule 5 of the challenge states that every LLM/AI call must go through Mesh API and that a submission which does not "will not be considered a valid submission." Adding Groq, OpenRouter, Together or any other provider SDK as a *runtime* path is therefore disqualifying, however cheap it is. Everything below stays inside Mesh.

1 — Resolve free models at runtime, never hardcode them. The free catalog changes; a hardcoded ID that gets delisted the night before judging is an avoidable failure. Query Mesh at startup, cache the result, and fall back down a preference chain.

```
# app/llm/model_router.py
from functools import lru_cache
from app.llm.mesh import client          # the single Mesh client

# ordered preference — first available wins
FREE_PREFERENCE = [
    "meta-llama/llama-3.3-70b-instruct",
    "qwen/qwen-2.5-72b-instruct",
    "deepseek/deepseek-chat",
    "mistralai/mistral-small",
    "google/gemma-2-9b-it",
]

PAID_FALLBACK = {"cheap": "openai/gpt-4o-mini",
                  "quality": "anthropic/claude-sonnet-4.5"}

@lru_cache(maxsize=1)
def free_model_ids() -> set[str]:
    # GET /v1/models — every model carries an is_free flag
    return {m.id for m in client.models.list() if getattr(m, "is_free", False)}

def resolve(node: str, tier: str) -> str:
    free = free_model_ids()
    for candidate in FREE_PREFERENCE:
        if candidate in free:
            return candidate
    return PAID_FALLBACK[tier]
```

Ship `scripts/list_free_models.py` (a three-line wrapper over the same call) and paste its output into the README. It proves the routing is real and it documents which models you were actually judged on.

2 — Lean on Mesh's own free response cache. Mesh provides a free 24-hour response cache plus provider-side prompt caching. Keep prompt prefixes stable — a fixed system prompt and catalog block, with only the user-behavior section varying — so the cacheable prefix is as long as possible.

3 — Set a spend cap on the key. Mesh supports per-key spend caps and rate limits. Cap the key at a few dollars: it makes runaway cost structurally impossible during a demo, and it is a line worth writing in the README under production thinking.

4 — The trigger policy is the real cost lever. The three-layer cache and drift trigger in this section matter more than model choice. At under one generation per 40 events, a demo session of 500 events is roughly a dozen generations — cents even on paid models.

5 — Develop against fixtures, not the API. `MESH_DISABLED=true` returns recorded fixtures for every node, so building the graph, the frontend and the tests costs nothing. If you want a live model while developing offline, run Ollama locally **behind the same flag** — it is a development convenience that never appears on the submitted code path, and the README should say so explicitly.

What to do with Groq and OpenRouter free tiers. They are genuinely useful and genuinely free within rate limits, but their place here is prompt prototyping in a scratch notebook outside the repo — not `app/`. Keep the repository single-gateway so a judge grepping for compliance finds exactly one client construction, in `app/llm/mesh.py`. A second provider SDK in `requirements.txt`

invites a reviewer to ask whether Mesh is really the path, which is a question you do not want asked.

6.6 Surfacing recommendations (F6)

`/dashboard` renders the current recommendation: headline, narrative, and product cards each carrying its "why this" line, plus a transparency strip — *"Based on 14 actions · 3 searches · updated 2m ago"* — with an expandable "what we noticed about you" panel showing top categories. Thumbs up/down writes a feedback event that feeds the next run. Skeleton loading, never a blocking spinner.

6.7 Scheduled proactive delivery (F7 — bonus)

APScheduler (`AsyncIOScheduler` with a Postgres jobstore so jobs survive restart):

- **09:00 daily** — for each opted-in user with activity in the last 24 hours, run the agent with `trigger_reason='scheduled'`, render a Jinja2 HTML email, send via SMTP or Resend. Optional Telegram bot as a second channel.
- **Every 5 seconds** — outbox drain.
- **03:00 daily** — vector drift audit, alerting the admin console on mismatch.

The digest is a recap with a hook: *"Yesterday you spent 20 minutes in advanced RAG. Here's the natural next step."* Include an unsubscribe path via `digest_optin`, and ship `scripts/send_digest_now.py` so you can demo it without waiting for 09:00.

6.8 Observability (F8 — bonus)

LangSmith tracing on every graph run under project `smartreco`, tagged with `user_id` and `trigger_reason`. Structured JSON logging with a `run_id` correlating API request to agent run to Mesh call. Admin page `/admin/agent-runs` lists every run with node path, retrieval score, refine loops, tokens, cost, latency, and a deep link into LangSmith. **Screenshot this in the README.**

7. Mesh API integration

A single client module, `app/llm/mesh.py`. Nothing else in the codebase constructs an LLM client — enforce it with one import-linting test, so a judge grepping for compliance finds exactly one integration point.

```
from openai import AsyncOpenAI
from app.core.config import settings

client = AsyncOpenAI(
    base_url="https://api.meshapi.ai/v1",
    api_key=settings.MESH_API_KEY,
)
```

Wrapper responsibilities: per-node model selection, `tenacity` retry with exponential backoff on 429 and 5xx, a 20-second timeout, token and cost accounting written to `agent_runs`, structured output

via Pydantic-validated JSON mode, and a `MESH_DISABLED=true` development flag that returns fixtures so you can build without burning quota.

`.env.example` is committed and lists every variable; `.env` is gitignored and holds the real ones. `MESH_API_KEY` and `SUBMISSION_TOKEN` go in GitHub repository secrets.

8. API surface

```
POST /api/auth/register {email, password, display_name}
POST /api/auth/login → sets httpOnly JWT cookie
GET /api/auth/me

GET /api/products ?category&level&q&page (q = hybrid semantic search)
GET /api/products/{id}
POST /api/admin/products [admin] → SQL + outbox in one transaction
PATCH /api/admin/products/{id} [admin]
DELETE /api/admin/products/{id} [admin]
GET /api/admin/sync-status [admin] → drift report
GET /api/admin/agent-runs [admin] → run telemetry
GET /api/admin/metrics [admin] → llm_calls_per_100_events, cache_hit_rate, cost

POST /api/events/batch → 202 Accepted, background bulk insert
GET /api/recommendations/current → stored current recommendation
POST /api/recommendations/refresh → force a run (rate-limited; demo affordance)
POST /api/recommendations/feedback {rec_id, item_id, signal}

GET /health → db, qdrant, redis, mesh reachability
```

9. Repository structure

smartreco/	
README.md	architecture, setup, bonus checklist, metrics
requirements.txt	fastapi, uvicorn, openai, langgraph, qdrant-client
.env.example	committed
.gitignore	MUST contain .env
docker-compose.yml	postgres + qdrant + redis, one command
.github/workflows/smartreco-checks.yml	
app/	
main.py	
core/	config, security, logging
db/	models, session, migrations/
api/routes/	auth, products, events, recommendations, admin
services/	catalog, outbox_worker, event_ingest, profile, trigger
vector/	qdrant_client, embeddings, hybrid_retriever, mmr, reranker
agent/	
graph.py	the LangGraph definition
nodes/	profile, plan, retrieve, grade, refine, generate, validate
state.py	
prompts/	
schemas.py	Pydantic structured outputs
llm/mesh.py	the ONLY LLM client in the repo
scheduler/	jobs.py, digest.py, templates/
frontend/	Next.js app/, components/, lib/tracker.ts
scripts/	seed_data.py, simulate_behavior.py, send_digest_now.py
evals/	eval_recommendations.py, dataset.json, results.md
tests/	test_dual_write.py, test_grounding.py, test_trigger.py, test_tracker_load.py

`scripts/simulate_behavior.py` is a secret weapon. It replays three scripted personas' event streams against a running instance, so a judge — or your demo video — watches recommendations visibly change in about 60 seconds with no manual clicking. Document it as the first thing under "try it" in the README.

10. The evaluation harness — a near-unique differentiator

A small offline suite over ten synthetic behavior profiles, scoring each generated recommendation on four axes.

Metric	Method	Target
Groundedness	Every recommended <code>product_id</code> exists in the catalog	100%
Behavioral relevance	Category overlap between recommendations and top interests	≥ 0.7
Persuasion quality	LLM-as-judge via Mesh, 1-5, on specificity and reference to real behavior	≥ 4.0
Diversity	Unique categories per recommendation set	≥ 2

Commit `evals/results.md` with the actual numbers. Almost no four-day hackathon submission measures its own output quality; doing so signals engineering maturity louder than any additional feature.

11. Delivery plan — 7 to 11 August

Day	Ship	Definition of done
Day 1 · Aug 7-8	Docker compose, schema and migrations, auth, product CRUD, outbox dual-write with drain worker, seed 40 courses	<code>sync-status</code> returns <code>in_sync: true</code> ; admin CRUD works end to end
Day 2 · Aug 9	Next.js catalog, product, search and dashboard shells; <code>tracker.ts</code> with batching and beacon; <code>/events/batch</code> ingest; profile builder; trigger evaluator	1,000 simulated events land correctly, profile updates, no UI jank
Day 3 · Aug 10	All seven LangGraph nodes, hybrid retrieval with MMR and rerank, Mesh routing, caching, grounding validator, dashboard rendering	Three personas produce three visibly different, grounded recommendations
Day 4 AM · Aug 11	APScheduler digest email, LangSmith tracing, <code>/admin/agent-runs</code> and metrics, evals run	Digest lands in inbox, traces visible, <code>results.md</code> committed
Day 4 PM · Aug 11	README with diagrams and metrics, demo video, final push, CI green	Critical checks pass, repo public

Cut order if you fall behind — drop bonuses in this sequence: Telegram → LangSmith → evals harness → email digest → reranker.

Never cut: dual-write integrity, the LangGraph loop, grounding validation, Mesh routing.

12. Risks and mitigations

Risk	Mitigation
A free-tier model is delisted before judging	<code>model_router.resolve()</code> walks a preference chain and falls back to a paid model; nothing hardcodes a single ID
Mesh rate limits or quota exhaustion mid-demo	Aggressive caching, <code>MESH_DISABLED</code> fixture mode, and pre-generated demo recommendations before recording
Next.js adds setup friction for judges	<code>docker-compose up</code> runs everything; also ship a minimal Jinja2 fallback page at <code>/simple</code> so the app is demonstrable backend-only
LangGraph refine loop runs away	Hard cap of two refine loops, a global 25-second timeout, and a deterministic fallback path
CI critical check fails on stray syntax errors	Keep all Python under <code>app/</code> , <code>scripts/</code> , <code>evals/</code> , <code>tests/</code> ; run <code>python -m compileall</code> locally before every push
Auth silently breaks once deployed cross-origin	Same-origin topology by default; if split, <code>SameSite=None; Secure</code> plus credentialed CORS, verified on the deployed URL
Daily digest never fires because the free host sleeps	GitHub Actions scheduled workflow calls a protected digest endpoint; documented in the README
A judge can't tell the agent is real	<code>/admin/agent-runs</code> , LangSmith links, and <code>evals/results.md</code> — three independent proofs

13. Deployment and demo

A deployed URL is optional and only reviewed for finalists — but it is cheap insurance, and it converts "here is my repo" into "here, click this." The whole stack fits inside free tiers.

13.1 Topology

Component	Host	Free-tier note
FastAPI backend + scheduler	Render web service (Docker) or Railway	Free instances sleep on idle — see §13.3
Next.js frontend	Vercel, or served from the same Render service	Vercel free tier is generous and zero-config
PostgreSQL	Neon or Supabase	Serverless Postgres, generous free row/storage caps
Qdrant	Qdrant Cloud free cluster, or a second container on the same host	1 GB free is far more than a 40-item catalog needs
Redis	Upstash	Free tier, REST or TCP; used only for cache and locks

Recommendation: put the frontend and backend on one host. Splitting them across Vercel and Render is the default instinct and it creates the cookie problem in §13.2 for no benefit at this scale. Either serve the Next.js build from the FastAPI container behind a single domain, or accept the cross-origin work deliberately.

13.2 The cross-origin cookie trap

The auth design in §6.1 puts the JWT in an httpOnly `SameSite=Lax` cookie. **Lax does not survive a cross-site request.** If the frontend is on `app.vercel.app` and the API is on `api.onrender.com`, every authenticated call — and every `sendBeacon` from the tracker — silently arrives without the cookie, and the whole demo looks broken in a way that is very hard to debug under time pressure.

Two valid resolutions:

- **Same origin (preferred).** One domain, no CORS, `SameSite=Lax` keeps working, tracking beacons just work.
- **Cross origin (if you must).** Set `SameSite=None; Secure` on the cookie, enable `CORSMiddleware` with `allow_credentials=True` and an explicit origin allow-list (a wildcard origin is rejected by browsers when credentials are enabled), and confirm `navigator.sendBeacon` fires with `credentials` — beacons send cookies only under these conditions.

Test this on the deployed URL, not on `localhost`, where every same-origin assumption holds and the bug is invisible.

13.3 The scheduler trap

The daily digest in §6.7 assumes a long-running process. **Free-tier web services sleep after idle, and serverless platforms have no persistent process at all** — APScheduler in-process will simply never fire at 09:00. Pick one:

- **A GitHub Actions scheduled workflow** hitting a token-protected `POST /api/internal/run-digest`. This is the pragmatic choice: it is free, it is visible in the Actions tab as proof it ran, and it survives the host sleeping. Note that GitHub's cron is best-effort and can drift by several minutes, which does not matter for a daily digest.
- **An external cron pinger** (cron-job.org or similar) against the same endpoint.
- **A paid always-on instance**, if you happen to have credit.

Keep APScheduler in the code for local runs and for the 5-second outbox drain — the drain must be in-process regardless, since it needs to be continuous. Document which mode the deployed instance uses; a judge who sees APScheduler in the repo and a sleeping host will assume the feature is fake unless you say otherwise.

13.4 Configuration and secrets

Everything in `.env.example` becomes a platform environment variable — never a committed file, and never baked into the image. The variables that change between local and deployed:

```
DATABASE_URL          # Neon connection string, ?sslmode=require
QDRANT_URL / QDRANT_API_KEY
REDIS_URL              # Upstash
MESH_API_KEY           # rsk...
MESH_DISABLED=false    # true only locally
JWT_SECRET             # rotate; never reuse the local dev value
COOKIE_SAMESITE         # lax | none – see §13.2
CORS_ORIGINS           # explicit list, no wildcard with credentials
FRONTEND_URL           # for links in digest emails
SMTP_* or RESEND_API_KEY
LANGSMITH_API_KEY / LANGSMITH_PROJECT
```

Run migrations on boot or as a release command, and seed the catalog once — a deployed instance with an empty catalog produces an agent that correctly retrieves nothing, which looks like a broken agent.

13.5 Pre-demo checklist

Run this against the deployed URL, not locally, before recording anything.

1. `GET /health` reports db, qdrant, redis and mesh all reachable.
2. Register a fresh user, log in, confirm the cookie is set and survives a reload.
3. Browse three courses; confirm events land (`SELECT count(*) FROM events`).
4. `GET /api/admin/sync-status` returns `in_sync: true`.
5. Trigger a recommendation; confirm it is grounded and `agent_runs` has a row with a full node path.
6. Fire `scripts/send_digest_now.py` ; confirm the email arrives.
7. Pre-generate recommendations for the three demo personas so the recording never waits on a cold LLM call.

13.6 Demo video script — 3 minutes

Optional, reviewed for finalists, and the highest-leverage three minutes you will spend if you have time.

Time	Beat
0:00–0:20	The problem in one sentence, then the dashboard with a personalized recommendation already on screen
0:20–0:50	Browse and search as a user; open devtools Network to show batched beacons, not one request per click
0:50–1:20	Return to the dashboard; the recommendation has changed. Expand "what we noticed about you"
1:20–1:50	Admin: add a product, then <code>sync-status</code> returning <code>in_sync: true</code> — the dual-write proof
1:50–2:30	<code>/admin/agent-runs</code> , then the LangSmith trace showing the refine loop firing
2:30–3:00	The digest email in an inbox, and the metrics table: LLM calls per 100 events

Lead with the behavior change and the grounding proof. Those are the two claims every other submission will make and few will demonstrate.

14. README outline — what judges read first

1. One-paragraph pitch plus a 30-second GIF of recommendations changing as behavior changes.
2. Architecture diagram.
3. Quickstart in under five commands: `cp .env.example .env` → `docker-compose up` → `python scripts/seed_data.py` → `python scripts/simulate_behavior.py` → open the dashboard.
4. Bonus checklist — LangGraph, scheduled email digest, LangSmith, hybrid retrieval with MMR and rerank — each linking to the exact file.
5. Efficiency section with the measured metrics table.
6. Design decisions and trade-offs: the outbox pattern, per-node model routing, why Postgres and Qdrant, what's out of scope and why.
7. Mesh API compliance, pointing at `app/llm/mesh.py` as the single integration point.
8. Evaluation results.
9. Deployment notes — the live URL, which scheduler mode it runs, and known free-tier limitations.

15. Rubric traceability

Brief requirement	Where it is satisfied
Login with two roles	§6.1
Clean, properly related schema	§5
Admin product CRUD	§6.2
Dual-write, stores stay in sync	§6.2 — outbox plus drift audit
Efficient non-blocking tracking	§6.3 — Web Worker, batching, throttle, sendBeacon
Sensible event schema	§5 — <code>events</code>
Agent consumes activity and reasons	§6.4 — nodes 1–2
RAG over vector DB, catalog-grounded	§6.4 — nodes 3 and 7
Persuasive personalized narrative	§6.4 — node 6
Stored and refreshed recommendations	§5 and §6.5
Don't call the LLM on every action	§6.5 — trigger policy plus three-layer cache
★ Structured agent framework	§6.4 — LangGraph, seven nodes, conditional edges
★ Scheduled proactive delivery	§6.7 — APScheduler daily digest
★ Observability	§6.8 — LangSmith plus <code>agent_runs</code>
★ Retrieval polish	§6.4 — hybrid, RRF, MMR, rerank, metadata filters
Mesh API mandatory	§7 — single client module
Optional deployed URL and demo video	§13 — topology, pre-demo checklist, video script